Lab Manual

of

Object Oriented Programming

Subject Code: 204ES Academic Year 2026–27



Name of Student : Pranshu Parikh
Student ID No. : 25EC419
Batch : A
Lab Faculty Name : Prof. Mayur Sevak

BACHELOR OF ENGINEERING

in

Electronics and Communication

BVM Engineering College
(An Autonomous Institution)
Vallabh Vidyanagar – 388120



BVM ENGINEERING COLLEGE
(An Autonomous Institution)
Vallabh Vidyanagar – 388120

Electronics and Communication Department
AY: 2026–27, Semester-III

CERTIFICATE

This is to certify that Mr./Ms. _____ of Year/Sem. _____, ID No. _____ has satisfactorily completed his/her term work in the month of _____ for the term ending in _____. Number of practicals certified _____ out of _____ in the subject of **204ES: Object Oriented Programming**.

Date: __ / __ / 2026

Signature of Teacher

BVM ENGINEERING COLLEGE

(An Autonomous Institution)
Vallabh Vidyanagar – 388120

Electronics and Communication Department AY: 2026–27, Semester-III

Object Oriented Programming (204ES)

INDEX

Name: Pranshu Parikh ID No.: 25EC419

Pr. No.	Title	Date	Sign
1.	Write a Program to display basic details of a User like Name, Roll no, Division and Address.		
2.	Write a Program to make simple calculator using Switch case.		
3.	Write a Program to Convert number of Days into Number of Months and remaining days.		
4.	Write a Program to enter marks of 6 subject and find out Total Marks and Percentage.		
5.	Write a Program to find whether the given number is even or odd.		
6.	Write a Program to find the largest among three numbers.		
7.	Write a Program to find whether given number is Armstrong or not.		
8.	Write a Program which generates table of a given number.		
9.	Write a Program which uses concept of Reference Variable.		
10.	Write a Program to swap two numbers using Call by Value method.		
11.	Write a Program to swap two numbers using Call by Reference Method.		
12.	Write a Program which uses Function as default argument.		
13.	Write a Program to calculate factorial of a number using recursion.		
14.	Write a Program to take input in an array and print its sum.		
15.	Write a Program to read string and print total number of characters.		
16.	Write a Program to create a class called 'Student' that has Name, ID, CPI, SPI and Semester as data Member. Also use proper member functions for taking and displaying information.		
17.	Write a Program to count number of objects using static variable.		
18.	Define a class to represent a bank account which includes the following data members: Name of Depositor, Account Number, Withdrawal Amount, Balance. Member functions: to assign initial value, to deposit an amount, to withdraw an amount after checking balance, and to display name and balance. Write the above program for handling n number of account holders using an array of objects.		
19.	Write a Program to demonstrate the use of Constructors and Destructors.		
20.	Write a Program which reads the complex number, copy that into other using copy constructor.		
21.	Write a Program to implement function Overloading for geometric area.		

Pr. No.	Title	Date	Sign
22.	Write a Program for addition of two complex numbers using Friend Function. (Use constructor function to initialize data members of complex class.)		
23.	Write a Program to find the mean value of given numbers using friend function.		
24.	Write a program to add integer value of two class objects.		
25.	Write a Program to do addition of two time objects.		
26.	Write a Program to design a class having static member function named showcount() which displays the number of objects created of the class.		
27.	Write a program which uses concept of Unary Operator (++/−) overloading.		
28.	Write a program which uses concept of Binary Operator overloading on String.		
29.	Write a program to create a Matrix and overload the '+' operator to add two matrix objects.		
30.	Define two classes Polar and Rectangle to represent a point in the polar and rectangular systems. Use conversion routines to convert from one system to the other.		
31.	Write a program to implement Single Inheritance.		
32.	Write a program to implement Multiple Inheritance.		
33.	Write a program to create File with constructor Function.		
34.	Write a program to demonstrate the use of Template.		

Program – 1

User Details Display

Aim:

Accept a user's Name, Roll No, Division and Address and print them as a neat summary.

Theory:

Standard input and output in C++ use the stream objects `cin` and `cout`. The extraction operator `>>` reads only up to whitespace, so multi-word text such as a full name or an address needs `getline()`. After a value is read with `cin`, the Enter key stays in the buffer, and `cin.ignore()` clears it so the next `getline()` does not return an empty string. The roll number here (25EC419) contains letters, so it is stored in a `string` instead of an `int`.

Algorithm:

1. Declare string variables for name, roll number, division and address.
2. Read the name with `getline()`.
3. Read the roll number with `cin`, then clear the leftover newline with `cin.ignore()`.
4. Read division and address with `getline()`.
5. Print every field with a label.

C++ Code:

```cpp
#include<iostream>
#include<string>
using namespace std;

int main()
{
    string name, roll, division, address;

    cout<<"Enter Name: ";
    getline(cin,name);

    cout<<"Enter Roll No: ";
    cin>>roll;
    cin.ignore();

    cout<<"Enter Division: ";
    getline(cin,division);

    cout<<"Enter Address: ";
    getline(cin,address);

    cout<<"\n------ Student Details ------";
    cout<<"\nName       : "<<name;
    cout<<"\nRoll No    : "<<roll;
    cout<<"\nDivision   : "<<division;
    cout<<"\nAddress    : "<<address;

    return 0;
}
```

Output:

```
Enter Name: Pranshu
Enter Roll No: 25EC419
Enter Division: EC-A
Enter Address: Anand

------ Student Details ------
Name       : Pranshu
Roll No    : 25EC419
Division   : EC-A
Address    : Anand
```

Conclusion:

All four details were captured and printed correctly. Combining `cin`, `getline()` and `cin.ignore()` is what keeps mixed numeric and text input from going wrong.

Program – 2

Calculator with Switch-Case

Aim:

Build a menu-style calculator for +, -, *, / and % driven by a `switch` statement.

Theory:

A `switch` compares one integer or character value against a list of constant labels and jumps straight to the matching one. Each branch is closed with `break` to stop execution from running into the next label, and `default` catches anything unexpected. For `/` and `%` the divisor must be tested for zero first; casting one operand to `float` keeps the fractional part of a quotient.

Algorithm:

1. Read two integers and an operator character.
2. Open `switch(op)` on the operator.
3. Compute the sum, difference or product in the matching case.
4. For `/` and `%` guard against a zero divisor before computing.
5. Use `default` to report an invalid operator.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num1, num2;
    char op;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter second number: ";
    cin >> num2;

    cout << "Enter operator (+, -, *, /, %): ";
    cin >> op;

    switch(op)
    {
        case '+':
            cout << "Addition = " << num1 + num2;
            break;

        case '-':
            cout << "Subtraction = " << num1 - num2;
            break;

        case '*':
            cout << "Multiplication = " << num1 * num2;
            break;

        case '/':
            if(num2 != 0)
                cout << "Division = " << (float)num1 / num2;
            else
                cout << "Division by zero is not allowed.";
```

```
            break;

        case '%':
            if(num2 != 0)
                cout << "Modulus = " << num1 % num2;
            else
                cout << "Modulus by zero is not allowed.";
            break;

        default:
            cout << "Invalid Operator!";
    }

    return 0;
}
```

Output:

```
Enter first number: 20
Enter second number: 10
Enter operator (+, -, *, /, %): +

Addition = 30

Enter first number: 25
Enter second number: 5
Enter operator (+, -, *, /, %): /

Division = 5

Enter first number: 15
Enter second number: 4
Enter operator (+, -, *, /, %): %

Modulus = 3
```

Conclusion:

Every operator gave the expected result, and zero divisors and unknown operators were handled gracefully. A `switch` is tidier than a long chain of `if` statements when one value is tested against many constants.

Program – 3

Days to Months Conversion

Aim:

Convert a count of days into whole months plus leftover days, taking a month as 30 days.

Theory:

Dividing one integer by another in C++ drops the fractional part, which gives the number of complete 30-day blocks. The modulus operator `%` supplies what is left over. Hence months $= \lfloor d/30 \rfloor$ and days left $= d \bmod 30$.

Algorithm:

1. Read the total number of days.
2. Find months with `days / 30`.
3. Find the remainder with `days % 30`.
4. Print both values.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int days, months, remainingDays;

    cout << "Enter the number of days: ";
    cin >> days;

    months = days / 30;
    remainingDays = days % 30;

    cout << "\nEquivalent Time:";
    cout << "\nMonths         = " << months;
    cout << "\nRemaining Days = " << remainingDays;

    return 0;
}
```

Output:

```
Enter the number of days: 75

Equivalent Time:
Months         = 2
Remaining Days = 15

Enter the number of days: 120

Equivalent Time:
Months         = 4
Remaining Days = 0

Enter the number of days: 49

Equivalent Time:
Months         = 1
Remaining Days = 19
```

Conclusion:

For 75 days the program reported 2 months and 15 days, as expected. Integer division together with the modulus operator splits a quantity into whole units and a remainder.

Program – 4

Marks Total and Percentage

Aim:

Read the marks of six subjects and report the total and percentage.

Theory:

The total is simply the sum of the six values. Taking each subject as out of 100, the percentage is the total divided by six. Dividing by `6.0` rather than `6` forces floating-point arithmetic, so a result like 83.33 is not cut down to 83.

Algorithm:

1. Declare six mark variables, plus `total` and `percentage`.
2. Read the six marks.
3. Add them into `total`.
4. Compute `percentage = total / 6.0`.
5. Print the total and the percentage.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int sub1, sub2, sub3, sub4, sub5, sub6;
    int total;
    float percentage;

    cout << "Enter marks of Subject 1: ";
    cin >> sub1;

    cout << "Enter marks of Subject 2: ";
    cin >> sub2;

    cout << "Enter marks of Subject 3: ";
    cin >> sub3;

    cout << "Enter marks of Subject 4: ";
    cin >> sub4;

    cout << "Enter marks of Subject 5: ";
    cin >> sub5;

    cout << "Enter marks of Subject 6: ";
    cin >> sub6;

    total = sub1 + sub2 + sub3 + sub4 + sub5 + sub6;

    percentage = total / 6.0;

    cout << "\nTotal Marks = " << total;
    cout << "\nPercentage = " << percentage << "%";

    return 0;
}
```

Output:

```
Enter marks of Subject 1: 85
Enter marks of Subject 2: 78
Enter marks of Subject 3: 90
Enter marks of Subject 4: 88
Enter marks of Subject 5: 76
Enter marks of Subject 6: 83

Total Marks = 500
Percentage = 83.33%
```

Conclusion:

The program returned a total of 500 and 83.33% for the sample marks. Choosing the right data type for the division is what keeps the percentage accurate.

Program – 5

Even / Odd Test

Aim:

Decide whether an entered integer is even or odd.

Theory:

Every even number leaves remainder 0 when divided by 2, and every odd number leaves remainder 1. The remainder operator `%` therefore turns the question into a single condition for an `if-else`.

Algorithm:

1. Read an integer.
2. Evaluate `num % 2`.
3. If the remainder is 0, print that the number is even.
4. Otherwise print that it is odd.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    if (num % 2 == 0)
        cout << num << " is Even.";
    else
        cout << num << " is Odd.";

    return 0;
}
```

Output:

```
Enter a number: 7
7 is Odd.

Enter a number: 12
12 is Even.
```

Conclusion:

Both even and odd inputs were classified correctly with one remainder test.

Program – 6

Largest of Three Numbers

Aim:

Find the greatest of three input numbers.

Theory:

A number is the largest if it is greater than or equal to each of the other two. Joining the two comparisons with the logical AND operator `&&` lets a single `if / else if / else` ladder cover every case, including ties.

Algorithm:

1. Read `a`, `b` and `c`.
2. Check whether `a` beats both others.
3. If not, check whether `b` beats both others.
4. Otherwise `c` is the largest.
5. Print the winner.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c;

    cout << "Enter three numbers: ";
    cin >> a >> b >> c;

    if (a >= b && a >= c)
        cout << "Largest number is: " << a;
    else if (b >= a && b >= c)
        cout << "Largest number is: " << b;
    else
        cout << "Largest number is: " << c;

    return 0;
}
```

Output:

```
Enter three numbers: 10 25 15
Largest number is: 25
```

Conclusion:

The correct maximum was printed for the test values. Logical operators let several relational checks be combined into one decision.

Program – 7

Armstrong Number

Aim:

Test whether a number equals the sum of its digits each raised to the power of the digit count.

Theory:

For a number with k digits, the Armstrong condition is $\sum d_i^k = N$; for instance $153 = 1^3 + 5^3 + 3^3$. The digits are peeled off one at a time using `% 10`, and the number is shortened using `/ 10`. The power is computed with `pow()` from `<cmath>`.

Algorithm:

1. Read the number and keep a copy.
2. Count its digits with a loop of repeated division by 10.
3. Loop again, adding each digit raised to the digit count to a running total.
4. Compare the total with the original number.
5. Print whether it is an Armstrong number.

C++ Code:

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int num, originalNum, remainder, digits = 0;
    int result = 0;

    cout << "Enter a number: ";
    cin >> num;

    originalNum = num;

    // Count the number of digits using for loop
    for (; originalNum != 0; originalNum /= 10) {
        digits++;
    }

    originalNum = num;

    // Calculate the sum of digits raised to the power of digits
    for (; originalNum != 0; originalNum /= 10) {
        remainder = originalNum % 10;
        result += pow(remainder, digits);
    }

    // Check Armstrong number
    if (result == num)
        cout << num << " is an Armstrong Number.";
    else
        cout << num << " is not an Armstrong Number.";

    return 0;
}
```

Output:

```
Enter a number: 371
371 is an Armstrong Number.
```

Conclusion:

371 was recognised as an Armstrong number. Looping with `%` and `/` is the standard way to work with the individual digits of an integer.

Program – 8

Multiplication Table

Aim:

Print the table of any entered number from 1 to 10.

Theory:

A `for` loop bundles the start value, the stopping test and the step into one line, which suits a task repeated a known number of times. Here the counter runs from 1 to 10 and each pass prints one line of the table.

Algorithm:

1. Read the number.
2. Run a loop with `i` from 1 to 10.
3. On every pass print `num x i = product`.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;

    cout << "\nMultiplication Table of " << num << ":\n";

    for (int i = 1; i <= 10; i++) {
        cout << num << " x " << i << " = " << num * i << endl;
    }

    return 0;
}
```

Output:

```
Enter a number: 7
Multiplication Table of 7:

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Conclusion:

The ten lines of the table were printed correctly, showing the `for` loop in its simplest form.

Program – 9

Reference Variable

Aim:

Show how a reference acts as a second name for an existing variable.

Theory:

A reference is declared with `&` (for example `int &ref = num;`) and must be bound to a variable at the moment it is created. It does not hold a copy: `ref` and `num` are two names for the same storage, so a change through either is seen through both.

Algorithm:

1. Read an integer into `num`.
2. Create `int &ref = num;`.
3. Print both to show they match.
4. Add 10 through `ref`.
5. Print both again to show `num` changed too.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main() {
    int num;

    cout << "Enter a number: ";
    cin >> num;

    // Reference variable
    int &ref = num;

    cout << "\nOriginal value of num = " << num << endl;
    cout << "Value using reference variable = " << ref << endl;

    // Modify the value using reference variable
    ref = ref + 10;

    cout << "\nAfter modifying through reference variable:" << endl;
    cout << "Value of num = " << num << endl;
    cout << "Value of ref = " << ref << endl;

    return 0;
}
```

Output:

```
Enter a number: 25
Original value of num = 25
Value using reference variable = 25

After modifying through reference variable:
Value of num = 35
Value of ref = 35
```

Conclusion:

Modifying the value through the reference also changed the original variable, confirming that both names share one memory location.

Program – 10

Swap by Value

Aim:

Attempt a swap using call by value and observe its limitation.

Theory:

When arguments are passed by value, the function receives its own copies. It can swap those copies freely, but the caller's variables stay as they were once the function ends.

Algorithm:

1. Read two numbers in `main()`.
2. Call `swap(a, b)` with plain `int` parameters.
3. Swap the local copies using a temporary variable and print them.
4. Print the original variables after the call.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Function using Call by Value
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;

    cout << "\nInside swap() function:" << endl;
    cout << "a = " << a << ", b = " << b << endl;
}

int main() {
    int x, y;

    cout << "Enter two numbers: ";
    cin >> x >> y;

    cout << "\nBefore swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;

    swap(x, y);

    cout << "\nAfter calling swap() function:" << endl;
    cout << "x = " << x << ", y = " << y << endl;

    return 0;
}
```

Output:

```
Enter two numbers: 10 20
Before swapping:
x = 10, y = 20

Inside swap() function:
a = 20, b = 10

After calling swap() function:
x = 10, y = 20
```

Conclusion:

The swap was visible only inside the function; `a` and `b` in `main()` were unchanged. Call by value therefore cannot alter the caller's data.

Program – 11

Swap by Reference

Aim:

Swap two numbers permanently using call by reference.

Theory:

If the parameters are declared as references (`int &x, int &y`), the function operates on the caller's own variables instead of copies. No values are copied and every change survives after the call.

Algorithm:

1. Read two numbers in `main()`.
2. Call `swap(a, b)` whose parameters are references.
3. Exchange them through a temporary variable.
4. Print the values in `main()` after the call.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Function to swap two numbers using call by reference
void swapNumbers(int &a, int &b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    int num1, num2;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter second number: ";
    cin >> num2;

    cout << "\nBefore Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;

    swapNumbers(num1, num2);

    cout << "\nAfter Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;

    return 0;
}
```

Output:

```
Enter first number: 25
Enter second number: 40

Before Swapping:
First Number = 25
Second Number = 40

After Swapping:
First Number = 40
Second Number = 25
```

Conclusion:

The values were exchanged in the calling function itself, which is the key difference from call by value.

Program – 12

Default Arguments

Aim:

Define a function whose parameters have default values.

Theory:

A default value is written in the function's parameter list. When the caller leaves out an argument, the default fills in; when it is given, the supplied value wins. Defaults must be assigned from the rightmost parameter towards the left.

Algorithm:

1. Define `add(int a = 10, int b = 20)`.
2. Call it with no arguments.
3. Call it with one argument.
4. Call it with two arguments.
5. Print each result.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Function with default arguments
int add(int a = 10, int b = 20)
{
    return a + b;
}

int main()
{
    cout << "add() = " << add() << endl;           // Uses both defaults
    cout << "add(5) = " << add(5) << endl;         // a = 5, b = 20
    cout << "add(5, 15) = " << add(5, 15) << endl; // a = 5, b = 15

    return 0;
}
```

Output:

```
add() = 30
add(5) = 25
add(5, 15) = 20
```

Conclusion:

One function handled three kinds of calls, falling back on the defaults whenever arguments were omitted.

Program – 13

Recursive Factorial

Aim:

Compute $n!$ with a function that calls itself.

Theory:

Recursion solves a problem by reducing it to a smaller copy of itself until a base case is reached. For the factorial, $n! = n \cdot (n-1)!$ and $0! = 1! = 1$ is the stopping point. The product is built as the calls return one by one.

Algorithm:

1. Read `n`.
2. In `fact()`, return 1 when `n` is 0 or 1.
3. Otherwise return `n * fact(n - 1)`.
4. Print the returned value.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Recursive function to calculate factorial
int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main()
{
    int num;

    cout << "Enter a number: ";
    cin >> num;

    if (num < 0)
    {
        cout << "Factorial of a negative number is not defined.";
    }
    else
    {
        cout << "Factorial of " << num << " = " << factorial(num);
    }

    return 0;
}
```

Output:

```
Enter a number: 5
Factorial of 5 = 120
```

Conclusion:

The recursive function returned the correct factorial; the base case is what prevents endless recursion.

Program – 14

Array Sum

Aim:

Store numbers in an array and add them.

Theory:

An array keeps several values of one type in consecutive memory, reached by an index that starts at 0. A loop fills the array from the keyboard, and a second loop adds each element to an accumulator that starts from zero.

Algorithm:

1. Read the element count `n`.
2. Fill the array using a loop.
3. Set `sum = 0` and add every element.
4. Print the sum.

C++ Code:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, sum = 0;

    cout << "Enter the number of elements: ";
    cin >> n;

    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    // Calculate sum of array elements
    for (int i = 0; i < n; i++)
    {
        sum = sum + arr[i];
    }

    cout << "Sum of array elements = " << sum << endl;

    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements:
10
20
30
40
50
Sum of array elements = 150
```

Conclusion:

The total of the entered elements was computed correctly by looping over the array.

Program – 15

String Length

Aim:

Read a line of text and report how many characters it contains.

Theory:

The `string` class from `<string>` offers the member function `length()`, which counts every character including spaces. `getline()` is used so that a whole sentence can be entered.

Algorithm:

1. Declare a `string`.
2. Read a full line with `getline()`.
3. Call `str.length()`.
4. Print the count.

C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string str;

    cout << "Enter a string: ";
    getline(cin, str);

    cout << "Total number of characters = " << str.length() << endl;

    return 0;
}
```

Output:

```
Enter a string: Hello World
Total number of characters = 11
```

Conclusion:

The character count for `Hello World` came out as 11, space included.

Program – 16

Student Class

Aim:

Create a `Student` class holding Name, ID, CPI, SPI and Semester, with functions to input and show them.

Theory:

A class groups data and the functions that work on it into a single type; an object is a concrete instance. Member functions reach the object's data directly, and are called through the dot operator. This packaging of data with behaviour is called encapsulation.

Algorithm:

1. Declare the class with the five data members.
2. Write `getData()` to read them.
3. Write `displayData()` to print them.
4. In `main()` create an object and call both functions.

C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student
{
private:
    string name;
    int id;
    float cpi, spi;
    int semester;

public:
    // Member function to accept student details
    void getData()
    {
        cout << "Enter Student Name: ";
        getline(cin, name);

        cout << "Enter Student ID: ";
        cin >> id;

        cout << "Enter CPI: ";
        cin >> cpi;

        cout << "Enter SPI: ";
        cin >> spi;

        cout << "Enter Semester: ";
        cin >> semester;
    }

    // Member function to display student details
    void displayData()
    {
        cout << "\n----- Student Information -----" << endl;
        cout << "Name      : " << name << endl;
```

```
        cout << "ID         : " << id << endl;
        cout << "CPI        : " << cpi << endl;
        cout << "SPI        : " << spi << endl;
        cout << "Semester   : " << semester << endl;
    }
};

int main()
{
    Student s;

    s.getData();
    s.displayData();

    return 0;
}
```

Output:

```
Enter Student Name: Rahul Patel
Enter Student ID: 101
Enter CPI: 8.75
Enter SPI: 9.10
Enter Semester: 5

----- Student Information -----
Name       : Rahul Patel
ID         : 101
CPI        : 8.75
SPI        : 9.1
Semester   : 5
```

Conclusion:

An object of the class accepted and displayed student records as designed, demonstrating a class with data members and member functions.

Program – 17

Object Counter (Static Member)

Aim:

Count how many objects of a class have been created.

Theory:

A static data member exists once per class, not once per object, so every object sees the same variable. It is declared in the class with `static` and defined once outside, for example `int Student::count = 0;`. Increasing it in the constructor records each new object.

Algorithm:

1. Declare `static int count` and define it outside the class.
2. Add one to `count` inside the constructor.
3. Create a few objects.
4. Print `count`.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Student
{
private:
    static int count;   // Static data member

public:
    // Constructor
    Student()
    {
        count++;
        cout << "Object " << count << " created." << endl;
    }

    // Static member function
    static void displayCount()
    {
        cout << "\nTotal number of objects created = " << count << endl;
    }
};

// Definition of static data member
int Student::count = 0;

int main()
{
    Student s1;
    Student s2;
    Student s3;

    Student::displayCount();

    return 0;
}
```

Output:

```
Object 1 created.
Object 2 created.
Object 3 created.

Total number of objects created = 3
```

Conclusion:

The shared counter matched the number of objects created, since all objects used the same static variable.

Program – 18

Bank Accounts with Object Array

Aim:

Model bank accounts with deposit and balance-checked withdrawal, for n customers.

Theory:

An array of objects holds many instances of a class, and element i is used as `acc[i].deposit()`. The `withdraw()` function compares the requested amount with the current balance and refuses the transaction if funds are short.

Algorithm:

1. Define `BankAccount` with name, account number, balance and withdrawal amount.
2. Add functions to assign values, deposit, withdraw and display.
3. Read `n` and declare an array of `n` objects.
4. Initialise each account, then run deposits and withdrawals.
5. Display name and balance for every account.

C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount
{
private:
    string name;
    int accountNo;
    float balance;
    float withdrawAmount;

public:
    // Function to assign initial values
    void assign()
    {
        cout << "\nEnter Depositor Name: ";
        cin.ignore();
        getline(cin, name);

        cout << "Enter Account Number: ";
        cin >> accountNo;

        cout << "Enter Initial Balance: ";
        cin >> balance;
    }

    // Function to deposit amount
    void deposit()
    {
        float amount;

        cout << "\nEnter amount to deposit: ";
        cin >> amount;

        balance = balance + amount;

```

```
38          cout << "Amount Deposited Successfully." << endl;
39      }
40
41      // Function to withdraw amount
42      void withdraw()
43      {
44          cout << "\nEnter amount to withdraw: ";
45          cin >> withdrawAmount;
46
47          if (withdrawAmount <= balance)
48          {
49              balance = balance - withdrawAmount;
50              cout << "Withdrawal Successful." << endl;
51          }
52          else
53          {
54              cout << "Insufficient Balance!" << endl;
55          }
56      }
57
58      // Function to display account details
59      void display()
60      {
61          cout << "\n-----------------------------" << endl;
62          cout << "Depositor Name : " << name << endl;
63          cout << "Account Number : " << accountNo << endl;
64          cout << "Balance        : " << balance << endl;
65      }
66  };
67
68  int main()
69  {
70      int n;
71
72      cout << "Enter Number of Account Holders: ";
73      cin >> n;
74
75      BankAccount acc[n];   // Array of Objects
76
77      // Accept details
78      for (int i = 0; i < n; i++)
79      {
80          cout << "\nEnter Details of Account Holder " << i + 1 << endl;
81          acc[i].assign();
82      }
83
84      // Deposit and Withdraw
85      for (int i = 0; i < n; i++)
86      {
87          cout << "\nTransactions for Account Holder " << i + 1 << endl;
88          acc[i].deposit();
89          acc[i].withdraw();
90      }
91
92      // Display details
93      cout << "\n\n----- Account Details -----" << endl;
94
95      for (int i = 0; i < n; i++)
96      {
97          acc[i].display();
98      }
99
100     return 0;
101 }
```

Output:

```
Enter Number of Account Holders: 2

Enter Details of Account Holder 1
Enter Depositor Name: Rahul Patel
Enter Account Number: 1001
Enter Initial Balance: 5000

Enter Details of Account Holder 2
Enter Depositor Name: Neha Shah
Enter Account Number: 1002
Enter Initial Balance: 8000

Transactions for Account Holder 1
Enter amount to deposit: 1000
Amount Deposited Successfully.
Enter amount to withdraw: 3000
Withdrawal Successful.

Transactions for Account Holder 2
Enter amount to deposit: 2000
Amount Deposited Successfully.
Enter amount to withdraw: 12000
Insufficient Balance!

----- Account Details -----

-----------------------------
Depositor Name : Rahul Patel
Account Number : 1001
Balance        : 3000

-----------------------------
Depositor Name : Neha Shah
Account Number : 1002
Balance        : 10000
```

Conclusion:

Balances changed correctly after deposits and valid withdrawals, and over-withdrawals were rejected. An object array makes handling many similar entities straightforward.

Program – 19

Constructor and Destructor

Aim:

Observe automatic initialisation and clean-up of objects.

Theory:

A constructor shares the class name, has no return type and runs on creation to initialise the object. The destructor, written with a leading `~`, runs when the object's life ends. Local objects are destroyed in the reverse order of their creation.

Algorithm:

1. Write a constructor that sets the data and prints a message.
2. Write a destructor that prints a message.
3. Create several objects in `main()`.
4. Display them and let the program end.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Student
{
public:
    // Constructor
    Student()
    {
        cout << "Constructor Called: Object Created." << endl;
    }

    // Destructor
    ~Student()
    {
        cout << "Destructor Called: Object Destroyed." << endl;
    }

    void display()
    {
        cout << "Welcome to C++ Programming!" << endl;
    }
};

int main()
{
    Student s1;   // Constructor is called automatically

    s1.display();

    return 0;     // Destructor is called automatically
}
```

Output:

```
Constructor Called: Object Created.
Welcome to C++ Programming!
Destructor Called: Object Destroyed.
```

Conclusion:

The constructor fired at creation and the destructor at the end of scope, in reverse order, exactly as theory predicts.

Program – 20

Copy Constructor

Aim:

Create one complex-number object as a copy of another.

Theory:

The copy constructor takes a reference to an existing object of the same class, `Complex(const Complex &c)`, and initialises the new object from it. It runs when an object is created from another one, passed by value or returned by value.

Algorithm:

1. Define `Complex` with a normal constructor and a copy constructor.
2. Read the real and imaginary parts and build the first object.
3. Build the second object with `Complex c2 = c1;`.
4. Display both.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Complex
{
    float real, imag;

public:
    // Parameterized constructor
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }

    // Copy Constructor
    Complex(const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }

    // Function to display complex number
    void display()
    {
        cout << real;

        if (imag >= 0)
            cout << " + " << imag << "i";
        else
            cout << " - " << -imag << "i";

        cout << endl;
    }
};

int main()
{
    float real, imag;
```

```
40
41      // Read complex number
42      cout << "Enter real part: ";
43      cin >> real;
44
45      cout << "Enter imaginary part: ";
46      cin >> imag;
47
48      // Create first object
49      Complex c1(real, imag);
50
51      // Copy c1 into c2 using copy constructor
52      Complex c2(c1);
53
54      cout << "\nOriginal Complex Number: ";
55      c1.display();
56
57      cout << "Copied Complex Number: ";
58      c2.display();
59
60      return 0;
61  }
```

Output:

```
Enter real part: 5
Enter imaginary part: 7

Original Complex Number: 5 + 7i
Copied Complex Number: 5 + 7i
```

Conclusion:

The second object held exactly the same value as the first, confirming that the copy constructor was used.

Program – 21

Overloaded Area Functions

Aim:

Compute areas of different shapes with functions sharing one name.

Theory:

Overloading lets several functions have the same name provided their parameter lists differ in number or type. The compiler chooses the right one from the arguments at the call, which is why this is called compile-time polymorphism.

Algorithm:

1. Write `area(radius)` for a circle.
2. Write `area(length, breadth)` for a rectangle.
3. Write `area(base, height, int)` for a triangle.
4. Read the dimensions, call each version and print the results.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Area of circle
float area(float radius)
{
    return 3.14159 * radius * radius;
}

// Area of rectangle
float area(float length, float breadth)
{
    return length * breadth;
}

// Area of triangle
float area(float base, float height, int)
{
    return 0.5 * base * height;
}

int main()
{
    float r, l, b, base, h;

    cout << "Enter radius of circle: ";
    cin >> r;
    cout << "Area of Circle = " << area(r) << endl;

    cout << "\nEnter length and breadth of rectangle: ";
    cin >> l >> b;
    cout << "Area of Rectangle = " << area(l, b) << endl;

    cout << "\nEnter base and height of triangle: ";
    cin >> base >> h;
    cout << "Area of Triangle = " << area(base, h, 0) << endl;

    return 0;
}
```

Output:

```
Enter radius of circle: 5
Area of Circle = 78.5397

Enter length and breadth of rectangle: 10 6
Area of Rectangle = 60

Enter base and height of triangle: 8 5
Area of Triangle = 20

#include <iostream>
using namespace std;
```

Conclusion:

The compiler picked the proper `area()` for each call, so one name served three different formulas.

Program – 22

Friend Function: Complex Sum

Aim:

Add two complex numbers with a friend function.

Theory:

A friend function is declared inside a class with `friend` but is not a member. It can still read the class's private data, and it is called without the dot operator. Here the friend receives two `Complex` objects and returns their sum.

Algorithm:

1. Define `Complex` with private parts and a constructor.
2. Declare `friend Complex add(Complex, Complex);`.
3. In `add()`, add real parts and imaginary parts separately.
4. Read two numbers, call `add()` and display all three.

C++ Code:

```
#include <iostream>
using namespace std;

class Complex
{
    float real, imag;

public:
    // Constructor to initialize data members
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }

    // Friend function declaration
    friend Complex add(Complex, Complex);

    void display()
    {
        cout << real << " + " << imag << "i";
    }
};

// Friend function definition
Complex add(Complex c1, Complex c2)
{
    Complex c3;

    c3.real = c1.real + c2.real;
    c3.imag = c1.imag + c2.imag;

    return c3;
}

int main()
{
    float r1, i1, r2, i2;

```

```
40      cout << "Enter real and imaginary part of first complex number: ";
41      cin >> r1 >> i1;
42
43      cout << "Enter real and imaginary part of second complex number: ";
44      cin >> r2 >> i2;
45
46      // Constructor initializes the objects
47      Complex c1(r1, i1);
48      Complex c2(r2, i2);
49
50      // Addition using friend function
51      Complex c3 = add(c1, c2);
52
53      cout << "\nFirst Complex Number = ";
54      c1.display();
55
56      cout << "\nSecond Complex Number = ";
57      c2.display();
58
59      cout << "\nAddition = ";
60      c3.display();
61
62      return 0;
63  }
```

Output:

```
Enter real and imaginary part of first complex number: 5 3
Enter real and imaginary part of second complex number: 4 2

First Complex Number = 5 + 3i
Second Complex Number = 4 + 2i
Addition = 9 + 5i
```

Conclusion:

The friend function accessed the private members of both operands and produced the right sum.

Program – 23

Friend Function: Mean

Aim:

Compute the mean of a set of numbers using a friend function.

Theory:

The numbers are kept in a private array. Because the friend function is granted access, it can total the elements directly and divide by the number of elements to get the mean.

Algorithm:

1. Define `Mean` with a private array and count.
2. Declare `calculateMean()` as a friend.
3. Read the numbers into the object.
4. Inside the friend, sum the values and divide by `n`.
5. Print the mean.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Mean
{
    int n;
    float arr[100];

public:
    Mean(int count)
    {
        n = count;
    }

    void getData()
    {
        cout << "Enter " << n << " numbers: ";
        for (int i = 0; i < n; i++)
            cin >> arr[i];
    }

    friend float calculateMean(Mean);
};

float calculateMean(Mean m)
{
    float sum = 0;

    for (int i = 0; i < m.n; i++)
        sum += m.arr[i];

    return sum / m.n;
}

int main()
{
    int n;

    cout << "Enter number of values: ";
```

```
40      cin >> n;
41
42      Mean m(n);
43      m.getData();
44
45      cout << "Mean = " << calculateMean(m);
46
47      return 0;
48  }
```

Output:

```
Enter number of values: 5
Enter 5 numbers: 10 20 30 40 50
Mean = 30
```

Conclusion:

The mean of the input values was calculated correctly through a friend function working on private data.

Program – 24

Adding Two Objects (+ Overload)

Aim:

Add the integer values held by two objects with the `+` operator.

Theory:

Operator overloading assigns a new meaning to an operator for a class. Defining `operator+` as a member means `a + b` is translated to `a.operator+(b)`: the left operand calls the function and the right operand is its argument.

Algorithm:

1. Define `Number` with an integer member.
2. Overload `+` to return a new object with the summed value.
3. Create two objects.
4. Add them with `c = a + b` and display the result.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Number
{
    int value;

public:
    // Constructor
    Number(int v = 0)
    {
        value = v;
    }

    // Operator overloading for +
    Number operator+(Number obj)
    {
        Number temp;
        temp.value = value + obj.value;
        return temp;
    }

    // Display function
    void display()
    {
        cout << "Sum = " << value << endl;
    }
};

int main()
{
    Number n1(10), n2(20), n3;

    n3 = n1 + n2;

    n3.display();

    return 0;
}
```

Output:

```
Sum = 30
```

Conclusion:

Adding two objects with the + symbol worked and gave the correct total.

Program – 25

Adding Two Time Objects

Aim:

Add two times given in hours, minutes and seconds.

Theory:

Time uses a base-60 system: 60 seconds make a minute and 60 minutes make an hour. After adding the fields, the overflow from seconds is carried into minutes and from minutes into hours. The `+` operator is overloaded to do this.

Algorithm:

1. Define `Time` with hours, minutes and seconds.
2. Read two times.
3. In `operator+`, add the seconds and carry any multiple of 60 into the minutes.
4. Do the same from minutes to hours.
5. Display the resulting time.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Time
{
    int hours, minutes, seconds;

public:
    void getTime()
    {
        cout << "Enter hours, minutes and seconds: ";
        cin >> hours >> minutes >> seconds;
    }

    void display()
    {
        cout << hours << " hours "
             << minutes << " minutes "
             << seconds << " seconds";
    }

    // Operator overloading for +
    Time operator+(Time t)
    {
        Time temp;

        temp.seconds = seconds + t.seconds;
        temp.minutes = minutes + t.minutes + temp.seconds / 60;
        temp.seconds = temp.seconds % 60;

        temp.hours = hours + t.hours + temp.minutes / 60;
        temp.minutes = temp.minutes % 60;

        return temp;
    }
};

```

```
int main()
{
    Time t1, t2, t3;

    cout << "Enter first time:\n";
    t1.getTime();

    cout << "Enter second time:\n";
    t2.getTime();

    t3 = t1 + t2;

    cout << "\nFirst Time = ";
    t1.display();

    cout << "\nSecond Time = ";
    t2.display();

    cout << "\nAddition of Two Times = ";
    t3.display();

    return 0;
}
```

Output:

```
Enter first time:
Enter hours, minutes and seconds: 5 45 50

Enter second time:
Enter hours, minutes and seconds: 3 25 30

First Time = 5 hours 45 minutes 50 seconds
Second Time = 3 hours 25 minutes 30 seconds
Addition of Two Times = 9 hours 11 minutes 20 seconds
```

Conclusion:

The two times were added with proper carries, giving a valid hours-minutes-seconds result.

Program – 26

Static Function showcount()

Aim:

Display the number of objects created, using a static member function.

Theory:

A static member function is tied to the class rather than to an object, so it can be called as `Demo::showcount()` without any object and can touch only static data. The counter is increased in the constructor.

Algorithm:

1. Declare `static int count` and `static void showcount()`.
2. Increase `count` in the constructor.
3. Create some objects.
4. Call `Demo::showcount()`.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Demo
{
    static int count;

public:
    Demo()
    {
        count++;
    }

    static void showcount()
    {
        cout << "Number of objects created = " << count << endl;
    }
};

// Definition of static data member
int Demo::count = 0;

int main()
{
    Demo d1;
    Demo d2;
    Demo d3;
    Demo d4;

    Demo::showcount();

    return 0;
}
```

Output:

```
Number of objects created = 4
```

Conclusion:

The static function reported the correct object count and was invoked through the class name.

Program – 27

Unary Operators ++ and --

Aim:

Overload the increment and decrement operators for a class.

Theory:

A unary operator has a single operand, so as a member function it takes no parameter: the object that calls it is the operand. `++n` becomes `n.operator++()` and updates the object's own data.

Algorithm:

1. Define `Number` with an integer member.
2. Overload `++` to increase it by one.
3. Overload `--` to decrease it by one.
4. Apply both operators on an object, displaying the value each time.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Number
{
    int value;

public:
    // Constructor
    Number(int v)
    {
        value = v;
    }

    // Unary ++ operator overloading
    void operator++()
    {
        ++value;
    }

    // Unary -- operator overloading
    void operator--()
    {
        --value;
    }

    // Display function
    void display()
    {
        cout << "Value = " << value << endl;
    }
};

int main()
{
    Number n(10);

    cout << "Before increment:" << endl;
    n.display();

```

```
41      ++n;
42
43      cout << "After increment:" << endl;
44      n.display();
45
46      --n;
47
48      cout << "After decrement:" << endl;
49      n.display();
50
51      return 0;
52  }
```

Output:

```
Before increment:
Value = 10
After increment:
Value = 11
After decrement:
Value = 10
```

Conclusion:

The value rose by one after `++` and fell back after `--`, proving both operators were overloaded correctly.

Program – 28

String Concatenation (+ Overload)

Aim:

Join two strings with an overloaded binary `+`.

Theory:

A binary operator has two operands; in a member-function overload the left one is the calling object and the right one is passed in. Overloading `+` for a `String` class makes `s3 = s1 + s2` perform concatenation.

Algorithm:

1. Define `String` to hold text.
2. Overload `+` to join the two texts into a new object.
3. Create two strings.
4. Add them and display the result.

C++ Code:

```cpp
#include <iostream>
#include <cstring>
using namespace std;

class String
{
    char str[100];

public:
    // Constructor
    String(const char s[] = "")
    {
        strcpy(str, s);
    }

    // Binary + operator overloading
    String operator+(String s)
    {
        String temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }

    // Display function
    void display()
    {
        cout << str << endl;
    }
};

int main()
{
    String s1("Hello ");
    String s2("World");

    String s3;

    s3 = s1 + s2;
```

```
40
41      cout << "First String  : ";
42      s1.display();
43
44      cout << "Second String : ";
45      s2.display();
46
47      cout << "Concatenated String : ";
48      s3.display();
49
50      return 0;
51  }
```

Output:

```
First String  : Hello
Second String : World
Concatenated String : Hello World
```

Conclusion:

Hello and World were joined into `Hello World` using only the `+` symbol.

Program – 29

Matrix Addition

Aim:

Overload `+` so two matrices can be added directly.

Theory:

Two matrices of the same order are added element by element: $C_{ij} = A_{ij} + B_{ij}$. The overloaded operator loops over every position, adds the pair, and stores the sum in a new matrix object.

Algorithm:

1. Define `Matrix` with a 2D array and read/display functions.
2. Overload `+` to add matching elements.
3. Read two matrices.
4. Compute `c = a + b` and display it.

C++ Code:

```cpp
#include <iostream>
using namespace std;

class Matrix
{
    int a[2][2];

public:
    // Function to input matrix elements
    void getdata()
    {
        cout << "Enter matrix elements:" << endl;

        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                cin >> a[i][j];
            }
        }
    }

    // Binary + operator overloading
    Matrix operator+(Matrix m)
    {
        Matrix temp;

        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                temp.a[i][j] = a[i][j] + m.a[i][j];
            }
        }

        return temp;
    }

    // Function to display matrix
    void display()
```

```
    {
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                cout << a[i][j] << "\t";
            }
            cout << endl;
        }
    }
};

int main()
{
    Matrix m1, m2, m3;

    cout << "Enter elements of first matrix:" << endl;
    m1.getdata();

    cout << "Enter elements of second matrix:" << endl;
    m2.getdata();

    // Add two matrix objects
    m3 = m1 + m2;

    cout << "\nFirst Matrix:" << endl;
    m1.display();

    cout << "\nSecond Matrix:" << endl;
    m2.display();

    cout << "\nResultant Matrix:" << endl;
    m3.display();

    return 0;
}
```

Output:

```
Enter elements of first matrix:
1 2
3 4

Enter elements of second matrix:
5 6
7 8

First Matrix:
1       2
3       4

Second Matrix:
5       6
7       8

Resultant Matrix:
6       8
10      12
```

Conclusion:

Matrix addition was expressed with the plain + operator and the sum matrix was correct.

Program – 30

Polar ↔ Rectangular

Aim:

Convert a point between polar and rectangular coordinates using two classes.

Theory:

Rectangular form (x, y) and polar form (r, θ) are linked by $r = \sqrt{x^2 + y^2}$, $\theta = \tan^{-1}(y/x)$, $x = r\cos\theta$ and $y = r\sin\theta$. A constructor that takes an object of the other class performs the conversion, and friendship between the classes lets it reach their private data.

Algorithm:

1. Define `Rectangle` and `Polar`, each with a conversion constructor.
2. Make the classes friends of each other as needed.
3. Create a rectangular point and convert it to polar.
4. Convert the polar point back and compare.

C++ Code:

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class Polar;

class Rectangle
{
    float x, y;

public:
    Rectangle(float a = 0, float b = 0)
    {
        x = a;
        y = b;
    }

    Rectangle(Polar p);       // Conversion constructor

    friend class Polar;       // Lets Polar access x and y

    void display()
    {
        cout << "Rectangular Coordinates = ("
             << x << ", " << y << ")" << endl;
    }
};

class Polar
{
    float r, theta;

public:
    Polar(float radius = 0, float angle = 0)
    {
        r = radius;
        theta = angle;
    }
```

```
39
40      Polar(Rectangle rec)      // Conversion constructor
41      {
42          r = sqrt(rec.x * rec.x + rec.y * rec.y);
43          theta = atan2(rec.y, rec.x) * 180 / 3.14159;
44      }
45
46      friend class Rectangle;
47
48      void display()
49      {
50          cout << "Polar Coordinates = ("
51               << r << ", " << theta << " degrees)" << endl;
52      }
53  };
54
55  Rectangle::Rectangle(Polar p)
56  {
57      float angle = p.theta * 3.14159 / 180;
58
59      x = p.r * cos(angle);
60      y = p.r * sin(angle);
61  }
62
63  int main()
64  {
65      Rectangle R1(3, 4);
66
67      cout << "Original Rectangle:\n";
68      R1.display();
69
70      // Rectangle -> Polar
71      Polar P1 = R1;
72
73      cout << "\nRectangle to Polar:\n";
74      P1.display();
75
76      // Polar -> Rectangle
77      Rectangle R2 = P1;
78
79      cout << "\nPolar to Rectangle:\n";
80      R2.display();
81
82      return 0;
83  }
```

Output:

```
Original Rectangle:
Rectangular Coordinates = (3, 4)

Rectangle to Polar:
Polar Coordinates = (5, 53.1301 degrees)

Polar to Rectangle:
Rectangular Coordinates = (3, 4)
```

Conclusion:

The point $(3, 4)$ became $(5, 53.13°)$ and returned to $(3, 4)$, so both conversions work.

Program – 31

Single Inheritance

Aim:

Derive one class from a single base class.

Theory:

Inheritance lets a derived class reuse the members of a base class. In `class Result : public Student`, the public and protected members of `Student` are available inside `Result`, which then adds its own.

Algorithm:

1. Define base class `Student` with roll number and name.
2. Define `Result` publicly derived from it, with marks.
3. Create a `Result` object and read all the data.
4. Display everything.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Base Class
class Student
{
protected:
    int rollNo;
    string name;

public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;

        cout << "Enter Name: ";
        cin >> name;
    }

    void displayStudentData()
    {
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};

// Derived Class
class Result : public Student
{
private:
    float marks;

public:
    void getResult()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
```

```
40
41      void displayResult()
42      {
43          displayStudentData();
44          cout << "Marks: " << marks << endl;
45      }
46  };
47
48  int main()
49  {
50      Result r;
51
52      r.getStudentData();
53      r.getResult();
54
55      cout << "\n--- Student Result ---\n";
56      r.displayResult();
57
58      return 0;
59  }
```

Output:

```
Enter Roll Number: 101
Enter Name: Sanket
Enter Marks: 85

--- Student Result ---
Roll Number: 101
Name: Rohan
Marks: 85
```

Conclusion:

The derived object used the base class's members along with its own, illustrating single inheritance and code reuse.

Program – 32

Multiple Inheritance

Aim:

Derive one class from two base classes.

Theory:

With multiple inheritance, the derived class lists several bases: `class Result : public Student, public Marks`. It then holds the members of all of them. If two bases define the same name, the use becomes ambiguous and must be qualified.

Algorithm:

1. Define base classes `Student` and `Marks`.
2. Define `Result` inheriting from both.
3. Read student data and marks through one object.
4. Display the combined result.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Base Class 1
class Student
{
protected:
    int rollNo;
    string name;

public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;

        cout << "Enter Name: ";
        cin >> name;
    }
};

// Base Class 2
class Marks
{
protected:
    float marks;

public:
    void getMarks()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
};

// Derived Class
class Result : public Student, public Marks
{
public:
```

```
40      void display()
41      {
42          cout << "\n--- Student Result ---" << endl;
43          cout << "Roll Number: " << rollNo << endl;
44          cout << "Name: " << name << endl;
45          cout << "Marks: " << marks << endl;
46      }
47  };
48
49  int main()
50  {
51      Result r;
52
53      r.getStudentData();
54      r.getMarks();
55
56      r.display();
57
58      return 0;
59  }
```

Output:

```
Enter Roll Number: 101
Enter Name: Naman
Enter Marks: 85

--- Student Result ---
Roll Number: 101
Name: Naman
Marks: 85
```

Conclusion:

Features of two separate classes were combined in one derived class through multiple inheritance.

Program – 33

File Creation in a Constructor

Aim:

Create and use a file whose opening is done by a constructor.

Theory:

File streams come from `<fstream>`: `ofstream` writes and `ifstream` reads. Opening the file inside the constructor ties its creation to the object's creation, and closing it in the destructor ties its release to the object's end.

Algorithm:

1. Give the class an `ofstream` member.
2. Open the named file in the constructor.
3. Write text with a member function.
4. Read the file back and print it.
5. Let the destructor close the file.

C++ Code:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class FileDemo
{
    ofstream fout;
    string fileName;

public:
    // Constructor creates (opens) the file
    FileDemo(string name)
    {
        fileName = name;
        fout.open(fileName.c_str());
        if (!fout)
            cout << "Error: File could not be created." << endl;
        else
            cout << "File '" << fileName << "' created successfully." << endl;
    }

    void writeData(string data)
    {
        fout << data << endl;
    }

    // Destructor closes the file
    ~FileDemo()
    {
        fout.close();
        cout << "File closed." << endl;
    }

    void readFile()
    {
        ifstream fin(fileName.c_str());
```

```
        string line;
        cout << "\nContents of the file:" << endl;
        while (getline(fin, line))
            cout << line << endl;
        fin.close();
    }
};

int main()
{
    string name, line;

    cout << "Enter file name: ";
    cin >> name;
    cin.ignore();

    FileDemo f(name);      // Constructor creates the file

    cout << "Enter a line of text: ";
    getline(cin, line);
    f.writeData(line);
    f.writeData("File handling using constructor in C++.");

    f.readFile();

    return 0;
}
```

Output:

```
Enter file name: notes.txt
File 'notes.txt' created successfully.
Enter a line of text: Hello from OOP lab

Contents of the file:
Hello from OOP lab
File handling using constructor in C++.
File closed.
```

Conclusion:

The file was created by the constructor, filled, read back and closed automatically by the destructor.

Program – 34

Templates

Aim:

Write generic code with a function template and a class template.

Theory:

A template is a blueprint in which the data type is a parameter, written `template <class T>`. The compiler produces a concrete version for every type it is used with, so one definition serves `int`, `float`, `char` and so on.

Algorithm:

1. Write `template <class T> T maximum(T a, T b)`.
2. Write the class template `Calculator<T>` with `add()` and `multiply()`.
3. Call `maximum()` for int, double and char.
4. Create `Calculator<int>` and `Calculator<float>` and print results.

C++ Code:

```cpp
#include <iostream>
using namespace std;

// Function template
template <class T>
T maximum(T a, T b)
{
    return (a > b) ? a : b;
}

// Class template
template <class T>
class Calculator
{
    T x, y;

public:
    Calculator(T a, T b)
    {
        x = a;
        y = b;
    }

    T add()      { return x + y; }
    T multiply() { return x * y; }
};

int main()
{
    cout << "Maximum of 10 and 25 = " << maximum(10, 25) << endl;
    cout << "Maximum of 3.5 and 2.8 = " << maximum(3.5, 2.8) << endl;
    cout << "Maximum of 'a' and 'z' = " << maximum('a', 'z') << endl;

    Calculator<int> c1(6, 4);
    cout << "\nInteger Calculator:" << endl;
    cout << "Sum = " << c1.add() << endl;
    cout << "Product = " << c1.multiply() << endl;

    Calculator<float> c2(2.5, 1.5);
```

```
40      cout << "\nFloat Calculator:" << endl;
41      cout << "Sum = " << c2.add() << endl;
42      cout << "Product = " << c2.multiply() << endl;
43
44      return 0;
45  }
```

Output:

```
Maximum of 10 and 25 = 25
Maximum of 3.5 and 2.8 = 3.5
Maximum of 'a' and 'z' = z

Integer Calculator:
Sum = 10
Product = 24

Float Calculator:
Sum = 4
Product = 3.75
```

Conclusion:

One template definition handled several data types, removing duplicated code while keeping type safety.